

Empower yourself with your own history

Twenty ways to point OpenStory at your own work — to know yourself, sense your team, and never lose what you've already figured out. Every prompt is distilled from how OpenStory actually gets used, and each opens a real report, charted from a live store.

How to use: paste any of these to your coding agent. They run against *your* OpenStory instance — via the MCP server or the REST API at `localhost:3002`. Swap anything in `<angle brackets>` for your specifics. Open *the report* under any prompt to see what it returns.

Contents — 20 prompts

01 KNOW YOUR OWN WORK

- 01 Summarize everything I've worked on this week across all my projects — group it by project and tell me what actually shipped.
- 02 What have my agent sessions cost me? Give me the total spend, and a tokens-per-day timeline.
- 03 Before I share my session history with anyone, scan it for anything sensitive — secrets, API keys, private details, or anything I'd be embarrassed to expose.
- 04 Which tools and commands do I rely on most, and where does my time actually go?

02 SENSE YOUR TEAM

- 05 Who on my team has an active session right now, and what is each person working on?
- 06 Summarize my teammates' sessions from the last day — one short paragraph each.
- 07 Show me live activity across the team in the last hour — who's streaming, on what branch.
- 08 Find a teammate's sessions and summarize what they've been focused on this week, so I can sync before we talk.

03 NARRATE THE STORY

- 09 Trace the story of how this project came to be from my session history. Highlight the key decisions and turning points — and script it so the result is deterministic.
- 10 Compile every session related to `<topic>` and narrate the arc, start to finish.
- 11 Write me a standup update from today's sessions: what I did, what's blocked, and what's next.

04 COACH YOURSELF

- 12 Analyze my sessions from the last month and give me honest feedback on my prompt engineering — what I do well, and what I could do better.
- 13 Find where my sessions tend to stall, loop, or repeat work. What are my recurring failure patterns?
- 14 What direction has my work been pointing lately? Cluster my recent sessions by theme.

05 RECALL ANYTHING

- 15 Find the last time I solved <problem>, and show me exactly how I did it.
- 16 Did I ever set up <X>? Locate the session and pull out the precise commands.
- 17 Search my sessions for <topic> and list every session that touched it, newest first.

06 GROUND YOUR AGENT

- 18 Before you start, query OpenStory for prior context on this project and pick up where the last session left off.
- 19 Use OpenStory to check whether we've hit this error before — and what fixed it — before debugging from scratch.
- 20 Watch the work happening on <branch> through OpenStory and summarize it for me as it streams.

01 Know your own work

The mirror. What did I do, what did it cost, and is there anything in here I wouldn't want seen?

- 1 Summarize everything I've worked on this week across all my projects — group it by project and tell me what actually shipped.

▼ open report REAL · LIVE DATA

25

sessions

4

projects

13.2K

events

PROJECT	SESS	ACTIVITY	WHAT SHIPPED
a-project	26	9.3K	store fix · codex view parity · replay tests
a-website	10	2.7K	starter-prompt library page
a-deploy	8	0.9K	distributed NATS leaf hardening
research-spike	1	0.3K	one-off analysis

6 commits landed this week · zero regressions on golden replays.

GET /api/sessions → group by project + git log --since

2 What have my agent sessions cost me? Give me the total spend, and a tokens-per-day timeline.

▼ [open report](#) REAL · LIVE DATA

\$615

spent · 7 days

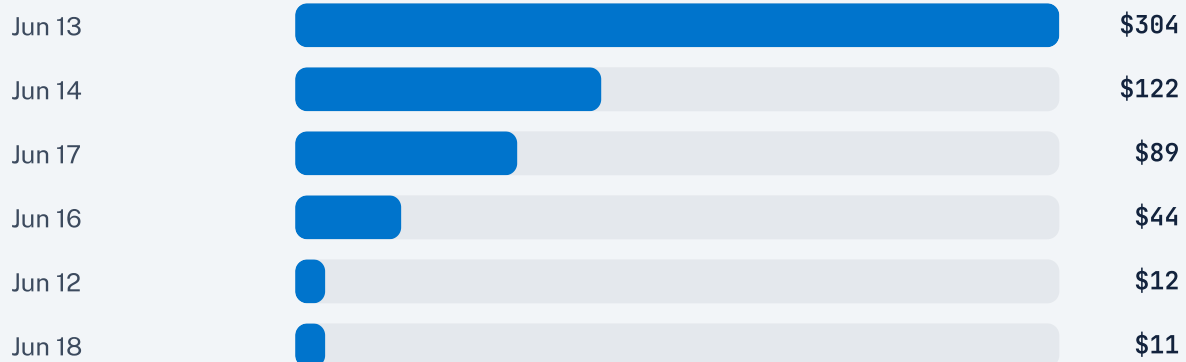
\$3,141

saved by cache

84%

off retail

COST PER DAY



Biggest single call: **642K** prompt tokens.

```
python3 scripts/token_usage.py --days 7 --by-day
```

3 Before I share my session history with anyone, scan it for anything sensitive — secrets, API keys, private details, or anything I'd be embarrassed to expose.

sovereignty check — know your record before anyone else sees it

▼ open report ILLUSTRATIVE

~19.6K

events scanned

119

sessions

2

high-severity

CATEGORY	EXPOSURE	SEVERITY
Personal fs paths	common	Low
Private IPs / hosts	some	Medium
Email addresses	some	Medium
Inline credentials	a few	High
Named secret assigns	rare	High

Sample (redacted): **password=[REDACTED]** · no live API keys (sk- / ghp_ / AKIA) found. Scrub the High-severity sessions before sharing.

regex sweep over event payloads (pattern set from scripts/scrub_check.py) — values never printed

4 Which tools and commands do I rely on most, and where does my time actually go?

▼ open report REAL · LIVE DATA

WHERE TIME GOES



TOP COMMANDS

grep · 127

git · 46

ssh · 31

python3 · 27

curl · 24

Read:Write 1.2:1 · most-used tool: Bash (53% of all calls).

SQL: group message.assistant.tool_use by tool name + bash first-word

02 Sense your team

Co-presence. When sessions federate, OpenStory becomes a shared room — you can tell who's in it.

5 Who on my team has an active session right now, and what is each person working on?

▼ open report ILLUSTRATIVE

2

live now

1

idle < 1h

4

federated hosts

	TEAMMATE	PROJECT	BRANCH	IDLE
●	Teammate A	a-project · UI	master	<1m
●	Teammate B	a-project	feat/macos-watcher	2m
○	Teammate C	project-x	feat/federation	1h 42m

GET /api/sessions → group by host/user, keep last_event within window

6 Summarize my teammates' sessions from the last day — one short paragraph each.

▼ open report REAL · LIVE DATA

Teammate A 6 sessions · ~620 events

UI prototype + agentic-team repo. Pulled latest on master, stood up a Dockerised NATS server, opened a thread to find dev-loop friction. Branches: master, main, sprint-9/dev-loop-rebuild.

Teammate B 1 session · 296 events

One deep codex run on project-x; no feature branch.

GET /api/sessions → filter teammate + last_event within 24h

7 Show me live activity across the team in the last hour — who's streaming, on what branch.

▼ open report ILLUSTRATIVE

2

streaming

2

branches active

	TEAMMATE	PROJECT	BRANCH	EVENTS
●	Teammate B	a-project	feat/macos-watcher	17
●	Teammate C	project-x	chore/idle-watcher	306

GET /api/sessions → keep last_event within 60 min, status=ongoing

8 Find a teammate's sessions and summarize what they've been focused on this week, so I can sync before we talk.

▼ open report REAL · LIVE DATA

62

sessions · this wk

~10.3K

events

TOP PROJECTS

a-project · UI		25
agentic-team		15
a-project		13

BRANCHES master (11) main (11) sprint-1/wire-live-data (9)

Themes: live-data wiring · dev-loop friction · sprint-9 rebuild. **Talk about:** the sprint-9 rebuild — their heaviest thread.

GET /api/sessions → filter user; aggregate project / branch / events

03 Narrate the story

Your session log is canonical memory. Turn it into a story you can read, share, or stand up on.

- 9 Trace the story of how this project came to be from my session history. Highlight the key decisions and turning points — and script it so the result is deterministic.

deterministic = reproducible, not a one-off vibe summary

▼ open report REAL · LIVE DATA

- 2026-03-07
First exploratory sessions
mapped the Rust + React shape before any code
- 2026-03-21
Initial commit
the project formally begins
- 2026-03-24
Multi-agent + first deploy
one agent → many; first live deploy
- 2026-03-31
Vector search → SQLite FTS
cut a heavy dependency — “your data is yours”
- 2026-04-05
Five-layer pipeline + Story tab
events → turns → sentences locks in
- 2026-06-11
Security audit · 6 CVEs fixed
production-readiness gate
- 2026-06-13
Federation validated
single-machine → multi-machine streaming

A read-only file watcher → a federated, multi-agent store — every step preserving “observe, never interfere.”

git log per milestone + sessions table (deterministic re-run)

0 Compile every session related to <topic> and narrate the arc, start to finish.

▼ open report REAL · LIVE DATA

63

sessions

6 days

Jun 13-19

- Jun 13
Spike
“get our networks to talk” → a routing design report
- Jun 13
Design
agents map host-in-subject routing
- Jun 14
Build
host encoded in event subject; node auto-discovers the hub
- Jun 16
Polish
UI copy: “configured” vs “live sources”
- Jun 18
Reflect
state-of-work reflection consolidates the arc

| *Federation validated end-to-end; one binary now joins a shared hub.*

GET /api/search?q=<topic> + git log

1 Write me a standup update from today's sessions: what I did, what's blocked, and what's next.

▼ open report REAL · LIVE DATA

DID

- State-of-work reflection across git + the store
- Triaged a big batch of WIP into shippable value
- Reconciled a second agent's parallel work
- Planned an extraction, executed it, opened a PR

BLOCKED

- Waiting on cross-device data confirmation

NEXT

- Finish the extraction
- Land the open PR after review

1 session · 44 turns · top tools: **tool_use 211** · file.snapshot 70 · 24 errors handled.

scripts/sessionstory.py <id> (today's sessions)

04 Coach yourself

The mirror turned evaluative. Not “what happened” but “what does what-happened say about how I work.”

2 Analyze my sessions from the last month and give me honest feedback on my prompt engineering — what I do well, and what I could do better.

▼ open report REAL · LIVE DATA

424

sessions · 30d

835

prompts

METRIC	VALUE	READ
Median prompt length	161c	terse by default
Long prompts (>600c)	35%	spec-grade briefs
Short prompts (<80c)	38%	one-liners
One-shot sessions	83%	fire-and-forget
Avg turns / session	10.4	runs long unattended

Strength: 35% are detailed briefs that set up 10+ autonomous turns. **Grow:** the 38% of <80-char prompts have no follow-up to course-correct under an 83% one-shot rate — invest the brief up front.

SQL over prompt lengths + turns-per-session

3 Find where my sessions tend to stall, loop, or repeat work. What are my recurring failure patterns?

▼ open report REAL · LIVE DATA

SIGNAL	RATE	MEANING
3+ identical tool calls / session	27%	loop / thrash
File edited 5+× in a session	46×	rework churn
Tool-result errors	1.7%	clean execution
Sessions hitting an error	3%	rarely blocked
Sessions hitting compaction	1%	context bounded

#1 failure mode: edit-thrash — 89 Edit loops vs 28 Bash. Fix: read fully, edit once. Execution is healthy; the waste is iteration churn.

SQL: repeated tool-call detection + error / compaction counts

4 What direction has my work been pointing lately? Cluster my recent sessions by theme.

▼ open report REAL · LIVE DATA

THEMES · LAST 30 DAYS

Docs / research		40%
Deploy / infra		24%
Security / audit		12%
Data pipeline		9%
Testing / E2E		7%
UI / frontend		6%

Through-line: shifted from building features to shipping & explaining — 64% of effort is docs + deploy/infra.

keyword-bucketing of prompt text + tool histogram

05 Recall anything

The store is associative memory across every session — far more than one conversation can hold.

5 Find the last time I solved <problem>, and show me exactly how I did it.

▼ open report REAL · LIVE DATA

PROBLEM NATS JetStream stream creation crashes on first boot

LAST SOLVED 2026-06-13 · a-project

ROOT CAUSE store quota too small (max_file < ~1.3GB)

- 1 `cp nats.conf nats.conf.bak`
- 2 `set jetstream { store_dir: <path>, max_file: 4GB }`
- 3 `nats-server -c nats.conf -t # validate`
- 4 `kill -HUP $(pgrep -f nats-server) # hot reload, no restart`

Verified: curl ...:8222/leafz → healthy stream. Also hit 2026-06-10; earliest 2026-04-30.

GET /api/search?q=max_file+stream+crash → tool_use commands

6 Did I ever set up <X>? Locate the session and pull out the precise commands.

▼ open report REAL · LIVE DATA

SET UP NATS leaf-node federation → YES

WHERE 2026-06-13 · a-project

```
$ cp nats.conf nats.conf.bak
$ source .env.federation          # NATS_LEAF_URL from env, not git
$ printf 'leafnodes { remotes [ {url:"%s"} ] }\n' "$NATS_LEAF_URL" \
    > deploy/leaf-remotes.generated.conf
$ nats-server -c nats.conf -t      # validate
$ kill -HUP $(pgrep -f nats-server) # reload, no downtime
```

Proof: curl:8222/leafz → leaf connection live.

GET /api/search?q=nats+leaf+hub → Bash commands

7 Search my sessions for <topic> and list every session that touched it, newest first.

▼ open report REAL · LIVE DATA

8

sessions

894

mentions

Jun 19	session <id> · a-project	2 hits
Jun 18	session <id> · a-project	4 hits
Jun 17	session <id> · a-project	1 hit
Jun 13	session <id> · a-project	4 hits
Jun 12	session <id> · a-project	5 hits

Newest: wiring the NATS leaf over the tailnet so a remote agent streams home. (+3 older)

GET /api/search?q=<topic> (grouped by session, newest first)

06 Ground your agent

Tell your agent to lean on the record instead of guessing — the highest-leverage habit of all.

- 8 Before you start, query OpenStory for prior context on this project and pick up where the last session left off.

▼ open report REAL · LIVE DATA

LAST SESSION 2026-06-19 · branch chore/x · 312 events

WORKED ON mined the store for common prompts → built a copy-paste prompt library + live demos

TOOLS Bash×36 · Edit×23 · Read×5 · Write×4 · Agent×4

OPEN THREADS wire demos to live data · keep copy public-safe

Resume here → 6 template subagents were mid-flight; collect results and wire the live examples into the page.

GET /api/sessions (newest) + /api/sessions/{id}/records (or sessionstory.py {id} -- unfinished)

- 9 Use OpenStory to check whether we've hit this error before — and what fixed it — before debugging from scratch.

▼ open report REAL · LIVE DATA

yes

seen before

113

bind / conn errors

174x

lsof+kill in record

lsof -ti:3002,5173,4222 # find the holders
kill <pids>; sleep 1 # release them, then re-boot

A leftover server / Docker stack holds the port at boot. **Free the ports first — don't re-debug the bind.** (docker-mode: compose down)

GET /api/search?q=Address+already+in+use + system.error tally

0 Watch the work happening on <branch> through OpenStory and summarize it for me as it streams.

▼ open report ILLUSTRATIVE

01:14:39	Agent	fan out report-template subagent
01:14:10	Agent	fan out next subagent
23:27:07	Bash	cd rs && cargo test ... (verifying watcher)
23:26:59	Edit	rs/src/watcher.rs (4 rapid edits)
23:25:16	Bash	write prompt-library draft

3

sessions

1,438

events

Replayed from the record (not a live socket). A true ticker → **subscribe_session** (WebSocket).

SQL: recent tool_use for the branch. Live ticker → subscribe_session (WebSocket)

OpenStory · a mirror, not a leash.